
Image Project

Foundamentals of Sound and Image

Authors:

Andrea Rodríguez Rodríguez

Diego Martínez Graña

Carlota Iglesias Martínez

Lucía Castrillo Sánchez

Tasks:

Task 4

Task 3

Task 2 and 5

Task 1, 6 and LaTeX report

May 8, 2026

Abstract

Optical Character Recognition (OCR) has been a fundamental technology for document digitization, information accessibility, and archival systems since the mid-20th century, as it converts printed or handwritten text in images into machine-readable text. Despite the rise of deep learning approaches, classical image processing methods remain relevant for controlled-domain applications where documents exhibit consistent structure, fixed fonts, and manageable lighting conditions. This project implements a complete end-to-end OCR pipeline for capital-letter documents using sequential image processing techniques in MATLAB.

The system architecture comprises six integrated stages: binarization (via Otsu's adaptive threshold method), vertical projection-based row detection and segmentation, horizontal projection-based character boundary detection, geometric normalization to fixed-size images, alphabet creation through reference document processing, and finally template-based character recognition using normalized 2D correlation.

Contents

1 Introduction

1.1 Project constraints and scope

This project focuses on a constrained but realistic scenario: documents containing only capital letters in Arial font, organized in horizontal rows, and scanned at reasonable resolution.

Despite the given constraints making the creation of the project easier, the implementation of the full pipeline from raw image input to final text output makes the project ambitious in breadth. This end-to-end approach reveals how errors at one stage propagate to downstream stages, a critical insight that single-stage misses.

Additionally, the project explores alternative methods at each stage: both base implementations (simpler, faster) and advanced variants (more robust, slower), both recommended by the guide and others self-discovered. At the end, informed trade-off decisions could be taken based on measured performance and will be later discussed.

1.2 Problem definition and success criteria

The overarching goal of this project is to build a functional OCR system that accurately recognizes capital letters in clean and moderately degraded documents, while thoroughly understanding and documenting the performance characteristics, failure modes, and improvement pathways of each component. This goal naturally decomposes into several specific objectives:

1. Implement a complete OCR pipeline that processes raw scanned images and outputs recognized text.
2. Achieve high recognition accuracy (targeting above 95%) on a test set of documents with varying levels of degradation.
3. Analyze the performance of each stage (binarization, segmentation, normalization, recognition) to identify bottlenecks and error sources.
4. Explore and compare alternative methods for each stage to understand their trade-offs in terms of accuracy, robustness, and computational efficiency.
5. Document the implementation details, results, and insights gained throughout the project in a comprehensive report.
6. Identify potential improvements and future work.

2 Methodology

The OCR system is organized as a linear pipeline designed with modularity as a core principle. Each of the six stages has a well-defined interface: it accepts input data, processes it deterministically, and returns output that feeds into other stage. This sequential organization reflects the logical flow of the OCR problem: we must first separate text from background (binarization), then locate where text regions are (row segmentation), then locate individual characters within those regions (character segmentation), then normalize those characters to a standard form (preprocessing), then establish what each character looks like (alphabet creation), and finally match observed characters against known templates (recognition).

The six stages are implemented as separate MATLAB functions, each with a clear purpose.

- Task 1 (`task1BinarizationAdvanced.m`) converts grayscale or RGB images into binary matrices.
- Task 2 (`task2ExtractRows.m`, `task2RowProjection.m`, `task2SegmentRows.m`) locates row boundaries.
- Task 3 (`task3SegmentCharacters.m`) locates character boundaries within rows.
- Task 4 (`task4PreprocessingCharacters.m`) resizes character images to 32×32 pixels.
- Task 5 (`task5CreatingAlphabet.m`) extracts and saves reference characters.
- Task 6 (`task6RecognizeCharacters.m`) matches test characters against alphabet templates.

2.1 Task 6 - Recognize characters

2.1.1 Recognition

The final stage of the system consists of identifying each segmented and preprocessed character by comparing it against a set of reference templates (alphabet) obtained in Task 5.

2.1.2 Methodology

Each input character image is first normalized to a fixed size of 32×32 pixels using the preprocessing pipeline described in Task 4. This ensures consistency between the input data and the stored templates.

The recognition process is based on template matching using a similarity metric. For each candidate character in the alphabet, a similarity score is computed between the

normalized input image X and the corresponding template T_i . In this implementation, the similarity is measured using the normalized two-dimensional correlation:

$$S_i = \text{corr2}(X, T_i) \quad (1)$$

where $S_i \in [-1, 1]$, with higher values indicating greater similarity.

The system evaluates all available templates in the alphabet structure, which is implemented as a MATLAB struct where each field corresponds to a character label (e.g., `alphabet.A`, `alphabet.B`, etc.).

The predicted character $\hat{c}$ is selected as the one that maximizes the similarity score:

$$\hat{c} = \arg \max_i S_i \quad (2)$$

2.1.3 Confidence Criterion

To improve robustness and avoid incorrect classifications, a confidence threshold is introduced. If the maximum similarity score is below a predefined threshold (empirically set to 0.5), the recognition is considered unreliable and the output is rejected:

$$\text{if } \max(S_i) < \tau \Rightarrow \hat{c} = "?" \quad (3)$$

This mechanism prevents low-confidence matches from being accepted as valid predictions.

Additionally, a margin-based criterion comparing the best and second-best scores can be introduced to further improve classification reliability.

2.1.4 Discussion

The use of normalized correlation provides a simple and computationally efficient approach to character recognition under controlled conditions. Since all characters share the same font and size, template matching is sufficient to achieve high accuracy.

However, this method presents several limitations. First, it is highly sensitive to segmentation errors and misalignments, as even small shifts can significantly reduce the correlation score. Second, the use of a single template per character does not capture intra-class variability, which may lead to incorrect recognition in the presence of noise or distortions.

To address these limitations, more advanced approaches could be considered, such as storing multiple templates per character, using normalized feature descriptors, or applying machine learning-based classifiers.

Despite these limitations, the implemented approach provides a solid baseline solution for the recognition task within the constraints of the problem.

EXEMPLO DE FIGURA A SER INSERIDA

Figure 1: Descrição da imagem

EXEMPLO DE REFERÊNCIA A SER INSERIDA

A Anexos 1.

B Anexos 2.